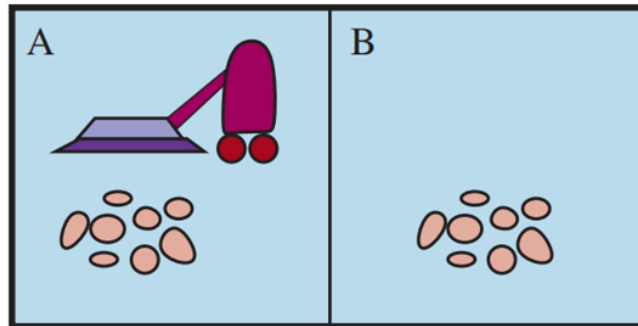


Consider a simple 1x2 grid world with two locations A and B. Locations can be dirty or clean. Agent starts in location A. The agent can sense the status of both locations A and B at any time. The agent has actions: Left, Right, Suck, NOOP. An example setting is illustrated in the figure below.



a) Give an efficient (compact) state representation of this problem. Briefly specify what each variable represents, domain, and initial value.

**Note1:** use math/set notation. Otherwise, no points will be awarded.

**Note2:** after giving your formulation, be sure to provide **domain** and **brief** descriptions for state variables so that it is clear to me what they represent and how searching with your formulation will work for this problem.

b) Using your state formulation, what is the initial state for the illustrated example?

c) Using your state formulation, define a goal test. You can give pseudocode. You can use python aggregators like *sum*, *max*, *min*, *all*, *any*.

You have an environment with  $N \times N$  grids (it is a 2D grid).  $N$  is given to you (integer). The agent can move up, down, left, right. The agent can paint the grid under him red or blue color. Goal is to paint half the grids blue and half the grids red. Assume  $N$  is even. At the start, the agent starts in top-leftmost grid position. All grids are uncolored. The agent can sense the status of all grids at all timesteps.

**a)** Give an efficient (compact) state representation of this problem. Briefly specify what each variable represents, domain, and initial value. Your answer should be for any  $N$ .

**Note1:** use math/set notation. Otherwise, no points will be awarded.

**Note2:** after giving your formulation, be sure to provide **domain** and **brief** descriptions for state variables so that it is clear to me what they represent and how searching with your formulation will work for this problem.

**b)** Using your state formulation, what is the initial state when  $N = 3$ ?

**c)** Using your state formulation, define a goal test. You can give pseudocode. You can use python aggregators like *sum, max, min, all, any*.

You have three jugs with capacities of 12 gallons, 8 gallons, and 3 gallons. You also have a water faucet. You can fill the jugs up or empty them out from one to another or onto the ground. The goal is to have a total of one gallon of water. Initially all jugs are empty.

**a)** Give an efficient (compact) state representation of this problem. Briefly specify what each variable represents, domain, and initial value.

**Note1:** use math/set notation. Otherwise, no points will be awarded.

**Note2:** after giving your formulation, be sure to provide **domain** and **brief** descriptions for state variables so that it is clear to me what they represent and how searching with your formulation will work for this problem.

**b)** Using your state formulation, what is the initial state?

**c)** Using your state formulation, define a goal test. You can give pseudocode. You can use python aggregators like *sum*, *max*, *min*, *all*, *any*.



Example with  $N = 8, K = 1, R = 1, M = 1$ .

1 ground-health at pos 3. 1 air-health at pos 8.

1 obstacle at pos 5.

Consider a single-line,  $N$  grid environment with grid positions  $1, 2, \dots, N$ . The agent has four actions: left, right, stay, and jump. The left and right actions advance agent position by one grid.

There are  $K$  grounded-health items positioned on the ground of a grid. There are  $R$  air-health items suspended in the air above a grid. There are  $M$  obstacles that occupy a grid that the agent cannot move through. The goal is to collect all ground and air health items.

For a problem instance of this environment, the agent knows the grid positions of the  $R$  ground-health items, the  $K$  air-health items, and  $M$  obstacles.

Ground health items are automatically grabbed by the agent once the agent moves to the item's grid. To be able to grab a health item in the air above the agent, the agent must be in the same grid of the item and stay for one timestep, and then jump in the next timestep. Finally, to be able to pass through an obstacle, the agent must move in the obstacles direction in one time step, and jump in the next timestep; this results in the agent advancing two grid positions and over the obstacle.

a) Give an efficient (compact) state representation of this problem. Briefly specify what each variable represents, domain, and initial value. Give your answer for any N, K, R, and M.

**Note1:** use math/set notation. Otherwise, no points will be awarded.

**Note2:** after giving your formulation, be sure to provide **domain** and **brief** descriptions for state variables so that it is clear to me what they represent and how searching with your formulation will work for this problem.

$$state = (agentloc, prevact, gh_1, gh_2, \dots, gh_K, ah_1, ah_2, \dots, ah_R)$$

where

$agentloc \in \{1, 2, \dots, N\}$  is the agent grid location

$prevact \in \{left, right, stay, jump\}$  denotes the action taken to get to this current state.

We check this in the result() function to determine whether the agent should grab an overhead air-health or jump over an obstacle.

$gh_i \in \{true, false\}$  denotes if ground-health collected. Initially all false.

$ah_i \in \{true, false\}$  denotes if air-health collected. Initially all false.

No need to track location of health and obstacles in state since they are static and coordinates given. They will be static lookup tables.

b) Using your state formulation, what is the initial state in the illustration?

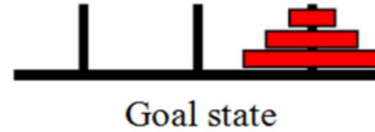
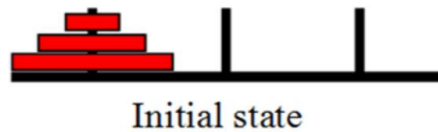
$$state = (1, stay, false, false)$$

The action value can be whatever action.

c) Using your state formulation, define a goal test. You can give pseudocode. You can use python aggregators like *sum*, *max*, *min*, *all*, *any*.

$$all(gh_1, gh_2, \dots, gh_K, h_1, ah_2, \dots, ah_R)$$

All these state variables need to be true.



Consider the Towers-of-Hanoi puzzle. It consists of three rods and an  $N$  number of disks of various diameters, which can slide onto any rod.

The puzzle begins with the disks stacked on one rod in order of decreasing size, the smallest at the top (see first figure).

The objective of the puzzle is to move the entire stack to one of the other rods, obeying the following rules:

- Only one disk may be moved at a time.
- Each move consists of taking the upper disk from one of the stacks and placing it on top of another stack or on an empty rod.
- No disk may be placed on top of a disk that is smaller than it.

The goal is to move the disks to the rightmost rod (see above right image).

**a)** Give an efficient (compact) state representation of this problem. Briefly specify what each variable represents, domain, and initial value. Answer should be for any  $N$ .

**Note1:** use math/set notation. Otherwise, no points will be awarded.

**Note2:** after giving your formulation, be sure to provide **domain** and **brief** descriptions for state variables so that it is clear to me what they represent and how searching with your formulation will work for this problem.

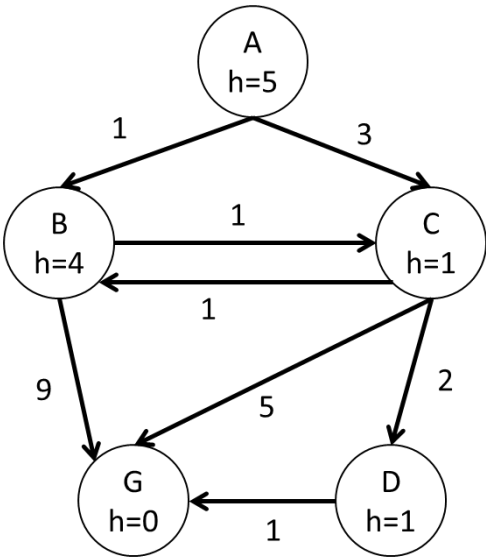
**b)** Using your state formulation, what is the initial state in the illustration?

**c)** Using your state formulation, define a goal test. You can give pseudocode. You can use python aggregators like *sum*, *max*, *min*, *all*, *any*.

Use best-first search template. Assume ties to be resolved in alphabetical order.  
Start node is S and goal node is G.  
**Show the search work and give the following.**

| Graph-like DFS Solution |  |
|-------------------------|--|
| Pop Order               |  |
| Solution Path           |  |
| Solution Cost           |  |
| # nodes popped          |  |
| # generated nodes       |  |

| Tree-like A* Solution |  |
|-----------------------|--|
| Pop Order             |  |
| Solution Path         |  |
| Solution Cost         |  |
| # nodes popped        |  |
| # generated nodes     |  |



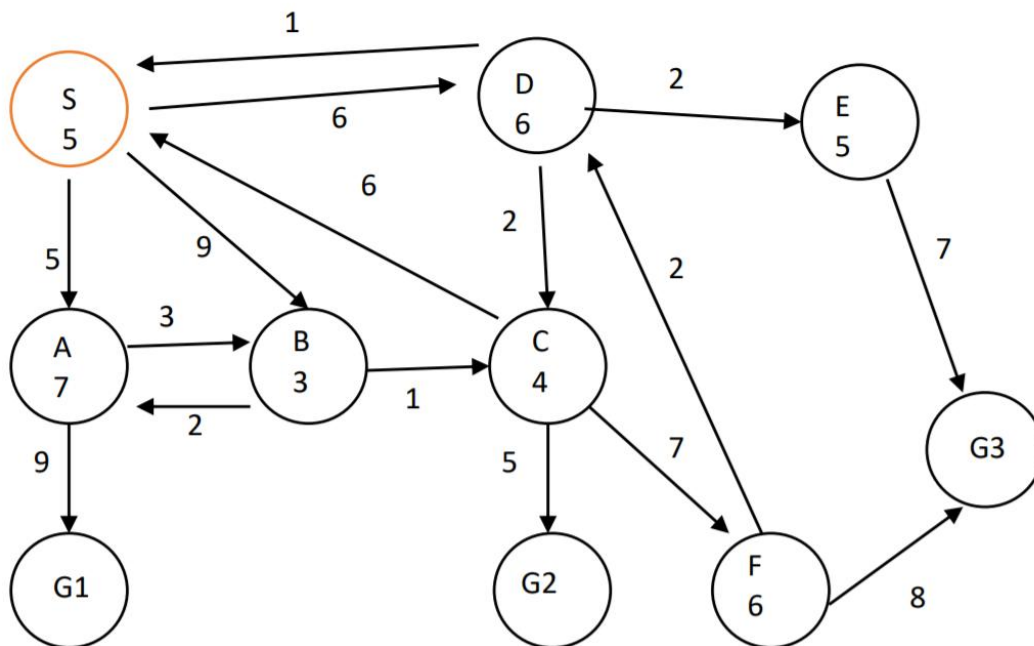
Is the given heuristic admissible? Why?

Is the given heuristic consistent? Why?

Make one change to the current heuristic to make it not admissible.

Apply BFS, DFS, UCS, Greedy, A\*. Use graph-like best-first search template. Assume ties to be resolved in alphabetical order. **Goal nodes are any one of {G1, G2, G3}**. For each algorithm give:

- Popped order.
- Solution path.
- Solution cost.
- Total nodes popped from frontier.
- Total nodes generated.

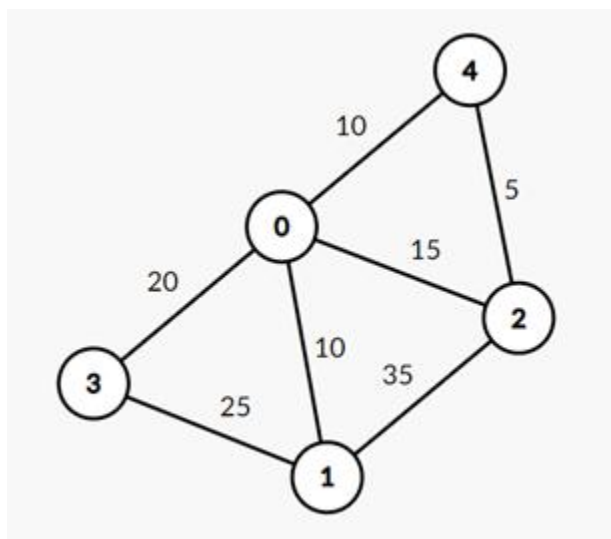


Is the heuristic given in the nodes admissible? Is it consistent?

## TSP Local Search [Formulation]

Consider a variant of the travelling-salesman problem. You are given an undirected graph  $G = (V, E)$  where  $V$  are the vertices and  $E$  are the edges. The edges have positive costs. The goal is to find the minimal-cost path where each vertex is visited exactly once. You can also choose the initial vertex and final vertex.

Assume that there are  $N$  vertex locations labelled as integers  $0, 1, 2, \dots, N - 1$ . Assume that the adjacency matrix  $\text{Adj}$  given is  $N \times N$  and it contains the edge costs. An example graph is given below.



We want to solve this problem using a local-search algorithm. Consider that we use simulated annealing to solve it.

- a) Give a suitable complete state formulation for this problem.

Note: use math notation.

$$\text{state} = (v_0, v_1, \dots, v_{N-1})$$

where each  $v_i \in \{1, 2, \dots, N\}$  represents a vertex location, and no two  $v_i$  can have the same value.

Thus a state represents a complete tour starting from  $v_0$  and ending  $v_{N-1}$ .

- b) Given the current state, propose a suitable **random-neighbor-generator**. Note, your proposal should make minor adjustments to the current state.

From a given state, we will randomly select two locations, and swap them. This can be done as follows:

1. Uniform-random sample two non-equal indices  $i$  and  $j$  from  $\{0, 1, \dots, N - 1\}$ .
2. Swap the locations at  $\text{state}[i]$  and  $\text{state}[j]$ .

c) Given a state, propose a suitable cost function that will help in solving this problem.

Given a state  $s$ , we use  $f(s)$  that computes the cost of the tour represented by  $s$ . So minimization problem. This can be done by looping on the state locations in a forward manner (from initial location to final location) and summing the edge costs from the adjacency cost matrix (cost from  $state[i]$  to  $state[i + 1]$  ).

```
tour_sum = 0
for i in range(N):
    tour_sum += adj_cost[state[i], state[i+1]]

return tour_sum
```

---

## Graph-coloring Local Search [Formulation]

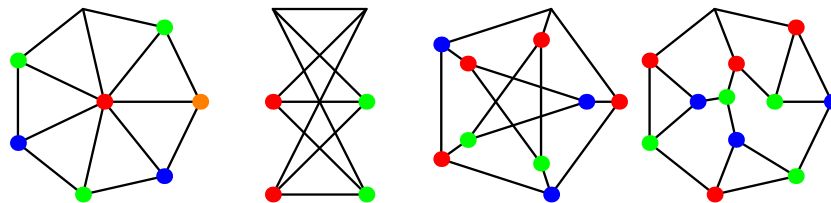


Image from link: [https://mathworld.wolfram.com/images/eps-svg/VertexColoring\\_750.svg](https://mathworld.wolfram.com/images/eps-svg/VertexColoring_750.svg)

Consider the graph-coloring problem (also known as vertex labelling). You are given a graph  $G = (V, E)$  where  $V$  are the vertices and  $E$  are the edges. There is a total of  $N$  vertices in  $G$ . There  $K$  possible colors to assign each vertex.

The goal is to assign a color to each vertex such that no two adjacent vertices have the same color. You can assume the vertices are labelled  $\{0, 1, 2, \dots, N - 1\}$  and colors are labelled from  $\{1, \dots, K\}$ .

We want to solve this problem using a local-search algorithm. You can assume simulated annealing is used.

d) Give a suitable complete state formulation for this problem.

Note: use math notation.

e) Given the current state, propose a suitable **random-neighbor-generator**. Note, your proposal should make minor adjustments to the current state.

f) Given a state, propose a suitable cost function that will help in solving this problem.

## SSP Local Search [Formulation]

Consider the subset-sum problem (SSP). We will work on a variant of it. You are given a set of non-negative integers  $S$ , and a target integer  $T$ . The goal is to find a subset  $B \subseteq S$  such that  $sum(B)$  equals to  $T$ . This problem is in NP, so no known efficient algorithm. However, we can model this problem as a local search problem and use a local search algorithm to try to find a solution (or approximately close solution).

When using a local search algorithm for this problem, the state should represent a subset that we can then sum over to check how good it is.

As an example, consider  $S = \{1, 54, 22, 43, 100\}$  and  $T = 122$ , then one possible subset is  $B = \{54, 22, 100\}$  with  $sum(B) = 176$

We want to solve this problem using a local-search algorithm. Consider that we use simulated annealing to solve it.

- a) Give a suitable complete state formulation for this problem.

Note: use math notation.

$$state = (e_1, e_2, \dots, e_{|S|})$$

Where  $e_i \in \{0,1\}$  indicates if  $i$ th element of  $S$  is in the subset state.

- b) Given the current state, propose a suitable **random-neighbor-generator**. Note, your proposal should make minor adjustments to the current state.

We have a uniform chance (50/50) of doing one of the following two actions:

- Remove an element from the *state* subset. This can be done by uniformly at random select the index of one of the 1 bits in *state*, and setting it to 0.  
 $state[rnd\_oneidx] = 0$
- Add an element to *state* subset. This can be done by uniformly at random selecting one of the 0 bits in *state*, and setting it to 1.  
 $state[rnd\_zeroidx] = 1$

- c) Given a state, propose a suitable cost function that will help in solving this problem.

Let  $B$  be the subset represented by *state*

$$f(B) = abs(T - sum(B))$$

The goal subset  $B$  achieves  $f(B) = 0$ . We favor subsets that make  $f(B)$  get closer and closer to 0. So, we are doing minimization